n-gram Language Model

CSEC 6 - Natural Language Processing

Statistical Language Processing

a subfield of natural language processing that applies probabilistic and statistical methods to analyze human language

Statistical Language Processing

- Statistical techniques can be used to disambiguate the input.
- They can be used to select the most probable solution.
- Statistical techniques depend on the probability theory.
- To able to use statistical techniques, we will need corpora to collect statistics.
- Corpora should be big enough to capture the required knowledge.

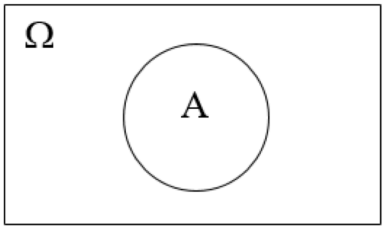
Review:
Basic Probability

Basic Probability

- **Probability Theory:** predicting how likely it is that something will happen.
- **Probabilities:** numbers between 0 and 1.
- **Probability Function:**
  - P(A) means that how likely the event A happens.
  - P(A) is a number between 0 and 1
  - P(A)=1 => a certain event
  - P(A)=0 => an impossible event
- **Example:** a coin is tossed three times. What is the probability of 3 heads?
  - 1/8
  - uniform distribution

Probability Spaces

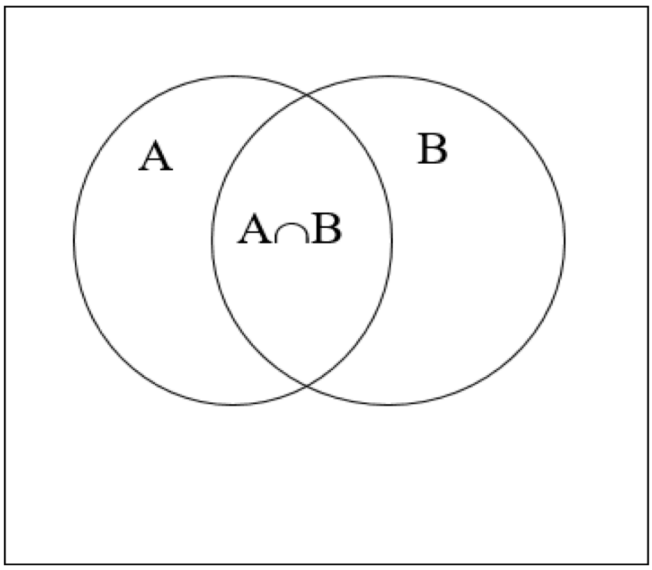
- There is a sample space and the subsets of this sample space describe the events.
- Ω is a sample space.
  - Ω is the certain event
  - the empty set is the impossible event.
- P(A) is between 0 and 1
- $P(\Omega) = 1$



Unconditional and Conditional Probability

- **Unconditional Probability or Prior Probability**
  - P(A): the probability of the event A does not depend on other events.
- **Conditional Probability-- Posterior Probability-- Likelihood**
  - P(A|B)**:** this is read as the probability of A given that we know B.
- Example:
  - P(put) is the probability of to see the word put in a text
  - P(on|put) is the probability of to see the word on after seeing the word put.
- **Joint Probability**
  - P(A∩B) or P(A,B): the probability of the events A and B occur together

Unconditional and Conditional Probability



$$P(A|B) = P(A \cap B) / P(B)$$

$$P(B|A) = P(A \cap B) / P(A)$$

Bayes' Theorem

- Bayes' theorem is used to calculate P(A|B) from given P(B|A).
- We know that:
  - P(A∩B) = P(A|B) P(B)
  - P(A∩B) = P(B|A) P(A)
- • So, we will have:

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)} \qquad P(B \mid A) = \frac{P(A \mid B)\, P(B)}{P(A)}$$

n-gram Language Model

Language Models

models that assign probabilities to sequences of words

n-gram Language Models

The simplest language model that assigns probabilities to sentences and sequences of words

n-gram

- An **n-gram** is a sequence of N words:
  - A **1-gram (unigram)** is a single word sequence of words like “please” or “ turn”.
  - A **2-gram (bigram)** is a two-word sequence of words like “please turn”, “turn your”, or ”your homework”.
  - A **3-gram (trigram)** is a three-word sequence of words like “please turn your”, or “turn your homework”.
- We can use n-gram models to estimate the probability of the last word of an n-gram given the previous words, and also to assign probabilities to entire word sequences.

Probabilistic Language Models

- **Probabilistic language models** can be used to assign a probability to a sentence in many NLP tasks.
    - In many NLP applications , we can use the probability as a way to choose a better sentence or word over a less- appropriate one.
- **Machine Translation:** P(high winds tonight) > P(large winds tonight)
- **Spell Correction:** “Thek office is about ten minutes from here”: P(The office is) > P(Then office is)
- **Speech Recognition:** P(I saw a van) >> P(eyes awe of an)
- **Summarization**, **question-answering**, ...

Probabilistic Language Models

- Our goal is to compute the probability of a sentence or sequence of words $W = (w_1, w_2, \ldots, w_n)$:
  - $P(W) = P(w_1, w_2, \ldots, w_n)$
- What is the probability of an upcoming word?:
  - $P(w_5 | w_1, w_2, w_3, w_4)$
- A model that computes either of these:
  - $P(W)$ or $P(w_n | w_1, w_2, \ldots, w_{n-1})$ is called a language model.

Chain Rule of Probability

- How can we compute probabilities of entire word sequences like $w_1, w_2, \ldots, w_n$?
  - The probability of the word sequence $w_1, w_2, \ldots, w_n$ is $P(w_1, w_2, \ldots, w_n)$.
- We can use the chain rule of the probability to decompose this probability:

$$\begin{aligned} P(w_1^n) &= P(w_1)\, P(w_2|w_1)\, P(w_3|w_1^2) \ldots P(w_n|w_1^{n-1}) \\ &= \prod_{k=1}^{n} P(w_k \mid w_1^{k-1}) \end{aligned}$$

- Example: P(the man from jupiter) = P(the) P(man|the) P(from|the man) P(jupiter|the man from)

Chain Rule of Probability and Conditional Probabilities

- The chain rule shows the link between computing the joint probability of a sequence and computing the conditional probability of a word given previous words.

- **Definition of Conditional Probabilities:**
  - $P(B|A) = P(A,B) / P(A) \rightarrow P(A,B) = P(A)\ P(B|A)$

- **Conditional Probabilities with More Variables:**
  - $P(A,B,C,D) = P(A)\ P(B|A)\ P(C|A,B)\ P(D|A,B,C)$

- **Chain Rule:**
  - $P(w_1 \ldots w_n) = P(w_1)\ P(w_2|w_1)\ P(w_3|w_1\ w_2) \ldots P(w_n|w_1 \ldots w_{n-1})$

Computing Conditional Probabilities

- To compute the exact probability of a word given a long sequence of preceding words is difficult (sometimes impossible).
- We are trying to compute $P(w_n|w_1 \ldots w_{n-1})$ which is the probability of seeing wn after seeing w_1^{n-1}.
- We may try to compute $P(w_n|w_1 \ldots w_{n-1})$ exactly as follows:

$$P(w_n|w_1 \ldots w_{n-1}) = \text{count}(w_1 \ldots w_{n-1}\ w_n) / \text{count}(w_1 \ldots w_{n-1})$$

- Too many possible sentences and we may never see enough data for estimating these probability values.
- So, we need to compute $P(w_n|w_1 \ldots w_{n-1})$ approximately.

n-grams

- The intuition of the n-gram model (simplifying assumption): instead of computing the probability of a word given its entire history, we can approximate the history by just the last few words.

$P(w_n \mid w_1 \ldots w_{n-1}) \approx P(w_n)$	**unigram**
$P(w_n \mid w_1 \ldots w_{n-1}) \approx P(w_n \mid w_{n-1})$	**bigram**
$P(w_n \mid w_1 \ldots w_{n-1}) \approx P(w_n \mid w_{n-1} w_{n-2})$	**trigram**
$P(w_n \mid w_1 \ldots w_{n-1}) \approx P(w_n \mid w_{n-1} w_{n-2} w_{n-3})$	**4-gram**
$P(w_n \mid w_1 \ldots w_{n-1}) \approx P(w_n \mid w_{n-1} w_{n-2} w_{n-3} w_{n-4})$	**5-gram**

- In general, N-Gram is

$$P(w_n \mid w_1 \ldots w_{n-1}) \approx P(w_n \mid w_{n-N+1}^{n-1})$$

n-grams

computing probabilities of word sequences

- Unigrams: $$\mathrm{P}(\mathrm{w}_1{}^{\mathrm{n}}) \approx \prod_{k=1}^{n} P(w_k)$$
- Bigrams: $$\mathrm{P}(\mathrm{w}_1{}^{\mathrm{n}}) \approx \prod_{k=1}^{n} P(w_k \mid w_{k-1})$$
- Trigrams: $$\mathrm{P}(\mathrm{w}_1{}^{\mathrm{n}}) \approx \prod_{k=1}^{n} P(w_k \mid w_{k-1} w_{k-2})$$
- 4-grams: $$\mathrm{P}(\mathrm{w}_1{}^{\mathrm{n}}) \approx \prod_{k=1}^{n} P(w_k \mid w_{k-1} w_{k-2} w_{k-3})$$

n-grams

computing probabilities of word sequences (Sentences)

- Unigram:
  - P(<s> the man from jupiter came </s>) ≈ P(the) P(man) P(from) P(jupiter) P(came)
- Bigram:
  - P(<s> the man from jupiter came </s>) ≈ P(the|<s>) P(man|the) P(from|man) P(jupiter|from) P(came|jupiter) P(</s>|came)
- Trigram:
  - P(<s> the man from jupiter came </s>) ≈ P(the|<s> <s>) P(man|<s> the) P(from|the man) P(jupiter|man f

n-gram models

- In general, a n-gram model is an insufficient model of a language because languages have long-distance dependencies.
  - “The computer(s) which I had just put into the machine room is (are) crashing.”
  - But we can still effectively use N-Gram models to represent languages.

n-gram models

- Bigger N, the model will be more accurate.
  - But we may not get good estimates for N-Gram probabilities.
  - The N-Gram tables will be more sparse.
- Smaller N, the model will be less accurate.
  - But we may get better estimates for N-Gram probabilities.
  - The N-Gram table will be less sparse.
- In reality, we do not use higher than Trigram (not more than Bigram).

n-grams and Markov Models

- The assumption that the probability of a word depends only on the previous word(s) is called Markov assumption.
- Markov models are the class of probabilistic models that assume that we can predict the probability of some future unit without looking too far into the past.
- A bigram is called a first-order Markov model (because it looks one token into the past);
- A trigram is called a second-order Markov model;
- In general a N-Gram is called a N-1 order Markov model.

Estimating n-gram Probabilities

- Estimating n-gram probabilities is called maximum likelihood estimation (or MLE).
- We get the MLE estimate for the parameters of an n-gram model by getting counts from a corpus, and normalizing the counts so that they lie between 0 and 1.
- Estimating bigram probabilities: (*where C is the count of that pattern in the corpus*)

$$P(w_n \mid w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)} = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

- Estimating N-Gram probabilities

$$P(w_n \mid w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}w_n)}{C(w_{n-N+1}^{n-1})}$$

Estimating n-gram Probabilities

- A mini-corpus: We augment each sentence with a special symbol <s> at the beginning of the sentence, to give us the bigram context of the first word, and special end-symbol </s>.

<s> I am Sam </s>
<s> Sam I am </s>
<s> I fly </s>

- Unique words: I, am, Sam, fly

Estimating n-gram Probabilities

- Bigrams: <s> and </s> are also tokens. There are 6(4+2) tokens and 6*6=36 bigrams

P(I|<s>)=2/3	**P(Sam|<s>)=1/3**	**P(am|<s>)=0**	**P(fly|<s>)=0**	P(<s>|<s>)=0	**P(</s>|<s>)=0**
P(I|I)=0	**P(Sam|I)=0**	**P(am|I)=2/3**	**P(fly|I)=1/3**	P(<s>|I)=0	**P(</s>|I)=0**
P(I|am)=0	**P(Sam|am)=1/2**	**P(am|am)=0**	**P(fly|am)=0**	P(<s>|am)=0	**P(</s>|am)=1/2**
P(I|Sam)=1/2	**P(Sam|Sam)=0**	**P(am|Sam)=0**	**P(fly|Sam)=0**	P(<s>|Sam)=0	**P(</s>|Sam)=1/2**
P(I|fly)=0	**P(Sam|fly)=0**	**P(am|fly)=0**	**P(fly|fly)=0**	P(<s>|fly)=0	**P(</s>|fly)=1**
P(I|</s>)=0	P(Sam|</s>)=0	P(am|</s>)=0	P(fly|</s>)=0	P(<s>|</s>)=1 or ?	P(</s>|</s>)=0

Estimating n-gram Probabilities

- **Unigrams:** I, am, Sam, fly

<s> I am Sam </s>
<s> Sam I am </s>
<s> I fly </s>

P(I)=3/8 P(am)=2/8 P(Sam)=2/8 P(fly)=1/8

- **Trigrams:** There are 6*6*6=216 trigrams.
  - Assume there are two tokens <s> <s> at the begining, and two tokens </s> </s> at the end.

P(I|<s> <s>)=2/3 P(Sam|<s> <s>)=1/3
P(am|<s> I)=1/2 P(fly|<s> I)=1/2
P(I|<s> Sam)=1
P(Sam|I am)=1/2 P(</s>|I am)=1/2
P(</s>|am Sam)=1
P(</s>|Sam </s>)=1

Thank you!

Text Processing

CSEC 6 - Natural Language Processing

Joseph Jessie S. Oñate, MSc.

Assistant Professor IV